

Real-Time Call Transcription

A Fresher's Complete Guide — From Raw Audio to Live Analysis

Criterion QA Engineering Team · March 2026 · v1.0

This guide explains — step by step, with real code — how a phone call is converted into text in real time, analysed for quality, and sent to supervisors, all within seconds of the words being spoken. Written for developers joining the team with no prior experience in audio processing or real-time systems.

1. The Big Picture — What Happens in 8 Seconds

When an agent picks up a phone and says "Thank you for calling CloudTech Support", here is exactly what happens inside our system:

Time	What Happens	Who Does It
0.0s	Agent speaks first word	Physical phone/headset
0.0s	Audio captured as PCM samples (8,000 per second)	PBX / SIP trunk
0.5s	First 500ms audio chunk sent to our server	Audio gateway (WebSocket)
0.7s	Chunk sent to Deepgram Nova-2 for transcription	Our Upload Service
1.1s	"Thank you for calling" returned as text + speaker	Deepgram API
1.2s	Partial transcript displayed on supervisor screen	WebSocket push to UI
8.0s	Full call end: LLM scores the complete transcript	Scoring Worker
8.3s	Quality scores, alerts sent to supervisor dashboard	Notification Service

2. Audio Fundamentals — What is "Audio" Really?

Before writing any code, you need to understand what audio actually is:

Sound wave → **numbers**. A microphone converts sound pressure into an electrical signal. An analogue-to-digital converter (ADC) samples that signal thousands of times per second, producing a stream of numbers called PCM (Pulse Code Modulation).

Property	Telephone call value	What it means
Sample rate	8,000 Hz (8 kHz)	8,000 numbers captured per second of audio
Bit depth	16-bit	Each number is between -32,768 and +32,767
Channels	1 (mono) or 2 (stereo)	Mono = both speakers mixed; stereo = separate channels
File format	PCM → compressed M4A/MP3/ RAW	Raw PCM is huge; formats compress it 10-20x
Data rate	~128 kbps	About 960 KB per minute of audio

Why does this matter?

Our system must handle audio in **chunks**, not as a single file. A 30-minute call = 1.7 GB of raw PCM. We cannot wait 30 minutes to start transcribing. Instead, we send audio in small pieces — called chunks — the moment they are recorded.

3. Chunking — How Audio is Sliced and Sent

A chunk is a small piece of audio, typically 500 milliseconds to 2 seconds long. Think of it like streaming a YouTube video — you do not download the whole video before it starts playing. The video arrives in small pieces called segments.

3.1 Fixed-size chunking (simpler, used in batch processing)

```
# APPROACH 1: Fixed-size chunks – split audio file into equal pieces
# Used when you have a complete file (e.g., post-call analysis)

from pydub import AudioSegment

def split_audio_into_chunks(audio_path: str, chunk_ms: int = 10000):
    """
    Split audio into chunks of chunk_ms milliseconds.
    10,000ms = 10 seconds per chunk.

    A 2-hour call at 10s chunks = 720 chunks processed in parallel.
    With 100 workers = ~8 seconds total (the 10s SLA is met!)
    """
    audio = AudioSegment.from_file(audio_path)
    chunks = []

    for i, start_ms in enumerate(range(0, len(audio), chunk_ms)):
        end_ms = min(start_ms + chunk_ms, len(audio))
        chunk = audio[start_ms:end_ms]

        # Add 500ms overlap to capture words split at boundaries
        if end_ms < len(audio):
            overlap = audio[end_ms:end_ms + 500]
            chunk = chunk + overlap

        # Export chunk to bytes (in memory – no disk I/O)
        buffer = io.BytesIO()
        chunk.export(buffer, format="wav")
        chunks.append({
            "chunk_id": i,
            "start_ms": start_ms,
            "end_ms": end_ms,
            "audio_data": buffer.getvalue(),
        })

    return chunks # returns list of dicts, one per chunk
```

3.2 Real-time streaming chunks (live calls)

For live calls, we cannot split a file because the file does not exist yet. Instead, the phone system sends audio continuously as a byte stream. We collect bytes for a short window (e.g., 500ms), then send that window to Deepgram:

```
# APPROACH 2: Real-time streaming – audio arrives continuously
# Used for live calls (the important production case)

import asyncio
import websockets

async def handle_live_call(websocket):
    """
    Called when an agent's phone connects via WebSocket.
    Audio arrives as raw bytes, 500ms at a time.
    We forward each chunk to Deepgram and push transcripts back.
    """
    # Connect to Deepgram streaming endpoint
    deepgram_ws = await websockets.connect(
        "wss://api.deepgram.com/v1/listen"
```


4. Transfer Protocol — How Chunks Move from Phone to Server

There are three ways chunks can travel from the phone system to our transcription server. Understanding which to use is one of the most important architectural decisions:

Protocol	How it works	Latency	Use case
HTTP POST (batch)	Upload complete file after call ends	Minutes	Post-call QA analysis (Criterion current)
WebSocket (streaming)	Persistent connection, continuous audio stream	~200ms	Live transcription (production target)
WebRTC (peer-to-peer)	Browser/app direct audio, no intermediate server	~50ms	Video/audio conferencing products
gRPC streaming (bidirectional)	Two-way stream, high throughput, low overhead	~100ms	Enterprise telco systems, Google STT
SIP/RTP (telephony)	Standard phone protocol, audio over UDP	< 20ms	PBX/call centre hardware integration

4.1 The WebSocket Streaming Flow (Production)

Here is the complete message sequence for a live call:

Step	Direction	Message	Who sends it
1	Phone → Server	CONNECT: call_id, agent_id, call_date	PBX / SIP gateway
2	Server → Phone	ACK: streaming_url, session_token	Our WebSocket server
3	Phone → Server	AUDIO_CHUNK (bytes, 500ms, repeating)	Phone system (continuous)
4	Server → Deepgram	Forward audio chunk	Our streaming worker
5	Deepgram → Server	Partial transcript + speaker + confidence	Deepgram API
6	Server → Browser	LIVE_TRANSCRIPT event (JSON)	Our WebSocket push
7	Phone → Server	CALL_END signal	PBX when call hangs up
8	Server → LLM	Full transcript for scoring	Our scoring worker
9	LLM → Server	Quality scores + feedback	Claude / GPT-4o
10	Server → Browser	CALL_COMPLETE with full results	Our WebSocket push

5. Interim vs Final Results — The Key to Feeling 'Live'

Deepgram returns two types of transcript events. Understanding the difference is what makes a transcription feel instant:

Type	What it is	When it arrives	Should you display it?
Interim result	Best guess so far — may change	Every 200-500ms	Yes — show in grey, updates as words arrive
Final result	Confirmed, will not change	When speaker pauses	Yes — show in black, permanent

Why this matters:

Without interim results: supervisor sees nothing, then suddenly a full sentence appears. **Latency feels high.** With interim results: supervisor sees words appearing letter by letter, like a live subtitle. **Latency feels zero.** Same underlying latency, completely different user experience.

How to handle interim vs final results in your frontend (JavaScript)

```
let partialTranscript = {}; // Store partial text per speaker
```

```
websocket.onmessage = (event) => {
```


6. Speaker Diarization — Who Said What?

Diarization is the process of identifying which speaker said which words. It is one of the hardest problems in speech recognition and is the reason we see '0 words - agent' bugs when it fails.

6.1 How Deepgram Nova-2 diarization works:

Nova-2 uses a neural model trained on millions of hours of two-speaker conversations. It analyses: voice pitch, speaking pace, silence patterns, and acoustic fingerprints. It labels each word with a speaker ID (0, 1, 2...) but does NOT know which is agent/customer — that mapping is our job.

6.2 What breaks diarization:

- **Crosstalk** — both speakers talking simultaneously confuses the model
- **Hold music** — gets labelled as a speaker, producing junk segments
- **Background noise** — a loud office can make one speaker sound like two
- **Mono recording of conference calls** — 3+ speakers on one channel is difficult

Production fix: Use stereo recording where possible. Contact-centre headsets record the agent on the LEFT channel and the customer on the RIGHT channel. This gives Deepgram channel-level separation — diarization accuracy goes from 85% to 99%.

7. The Complete Real-Time Pipeline — All Pieces Together

8. Latency — Why Does It Take 8 Seconds?

Understanding where time is spent is critical to debugging and optimisation:

Stage	Time	Why	How to reduce
Audio transmission	0 – 50ms	Network from phone to our server	Use data centres near call origin (CDN edge nodes)
Audio buffering	0 – 500ms	We collect 500ms before sending to Deepgram	Reduce to 200ms (trade-off: more API calls)
Deepgram processing	200-600ms	Nova-2 neural model inference	Use nova-2-phonecall model (optimised for telephony)
Network to Deepgram	10-100ms	Data centre to Deepgram API	Use Deepgram self-hosted deployment (\$\$\$)
LLM scoring	2-8s	Claude needs full transcript to score	Stream partial transcript; score mid-call for 30-min calls
Database write	10-50ms	PostgreSQL insert	Use async writes; never block the main thread on DB
WebSocket push to UI	1-10ms	JSON serialisation + network	Use binary MessagePack instead of JSON (3x faster)
Total (live transcript)	< 800ms	Audio spoken to text on screen	Target: < 400ms with optimised chunking
Total (full scoring)	3 – 8s	Call end to scores on dashboard	Target: < 5s with Bedrock Claude Instant

9. What Can Go Wrong — Error Handling for Freshers

Real systems fail. Here are the 5 most common failures and how to handle them:

Error: Network drop mid-call
Symptom: WebSocket disconnects while call is live
Fix: 1. Detect disconnect (<code>websocket.on_close</code>) 2. Buffer the last 2s of audio 3. Try reconnect 3 times 4. If failed: save buffered audio to S3 5. Resume transcription from S3 when reconnected
Error: Deepgram API rate limit
Symptom: Too many concurrent streaming connections
Fix: 1. Monitor active connections (Gauge metric) 2. Queue new calls if > 80% of limit 3. Scale horizontally: each worker handles N connections 4. Set up Deepgram Enterprise with dedicated limit
Error: LLM timeout
Symptom: Claude API takes > 30s to respond
Fix: 1. Set strict 25s timeout on all LLM calls 2. Fall back to rule-based scoring (always available) 3. Retry once in background 4. Never block the UI waiting for LLM
Error: Empty transcript
Symptom: Deepgram returns 0 words
Fix: 1. Log the audio file path for investigation 2. Return empty transcript with error flag set 3. Show user: 'Transcription failed – check API key' 4. Fire System alert (not Agent alert – this is infrastructure)

Error: Speaker diarization wrong

Symptom: Agent and customer labels are swapped

Fix:

1. Add manual override button in UI
2. Log calls where agent spoke 0 words (impossible)
3. Use stereo audio recording to prevent this
4. Human review queue for flagged calls

10. Scaling — Handling 10,000 Simultaneous Calls

A single Python process can handle roughly 100-200 concurrent WebSocket connections. At 10,000 simultaneous calls, you need a distributed architecture:

```
# SCALING ARCHITECTURE — simplified

# PROBLEM: 10,000 calls at once = 10,000 WebSocket connections
# SOLUTION: Horizontal scaling with a load balancer

# Layer 1: API Gateway / Load Balancer
#   → Routes each call to a healthy WebSocket worker
#   → Sticky sessions (call sticks to one worker for its lifetime)
#   → Health checks remove crashed workers automatically

# Layer 2: WebSocket Workers (Kubernetes pods)
#   → Each pod handles 200 concurrent calls
#   → 10,000 / 200 = 50 pods needed
#   → Kubernetes autoscaling adds pods when queue depth > threshold

# Layer 3: Message Queue (Kafka / SQS)
#   → When a call ends, push "call_ended" event to queue
#   → Multiple scoring workers consume events independently
#   → If one scorer crashes, the event is retried automatically

# Layer 4: Scoring Workers (separate pods)
#   → Pull call_ended events from queue
#   → Call LLM, store results, fire alerts
#   → Scale independently from WebSocket workers (LLM is slow)

# Pseudocode for Kubernetes HPA (Horizontal Pod Autoscaler):
#
# - When active_calls / total_capacity > 70%: add 5 WebSocket worker pods
# - When LLM queue depth > 100: add 3 scoring worker pods
# - When idle for 10 minutes: scale down to minimum
```

11. Your First Task — A 30-Minute Exercise

To test your understanding, implement the following minimal real-time transcription system. You should be able to complete this in one day:

Step	Task	Estimated Time
1	Install: pip install deepgram-sdk websockets pydub flask	15 min
2	Create a Flask WebSocket server (use flask-sock) that accepts audio bytes	30 min
3	Inside the WebSocket handler, forward audio chunks to Deepgram streaming API	45 min
4	Print each Deepgram response to the console — see transcripts appear live	15 min
5	Map speaker_0 to 'Agent' and speaker_1 to 'Customer' using first-to-speak rule	15 min
6	Broadcast the transcript to a second WebSocket (the 'supervisor' screen)	30 min
7	Record a 60-second audio file with yourself playing both roles, run it through	20 min
8	Observe latency: how many seconds between speaking and seeing text on screen?	10 min

Glossary — Terms You Will Hear Every Day

Term	Plain English Definition
------	--------------------------

PCM	Raw audio numbers — the most basic form of digital audio
Diarization	Identifying who is speaking at each moment
Utterance	One continuous piece of speech by a single speaker
Interim result	Deepgram's best guess before the speaker finishes their sentence
Final result	Deepgram's confirmed, locked transcript segment
WebSocket	A persistent connection that can push data in both directions (unlike HTTP)
Chunking	Splitting a large file or stream into small pieces for processing
SIP	Session Initiation Protocol — the language telephones speak to each other
PBX	Private Branch Exchange — the telephone switchboard in an office
LLM	Large Language Model — AI model (Claude, GPT-4) that reads text and produces text
RAG	Retrieval-Augmented Generation — LLM answers that include facts from your documents
Latency	The delay between something happening and you seeing the result
Throughput	How many things (calls, chunks, transcripts) the system handles per second
Horizontal scaling	Adding more computers instead of making one computer more powerful
Idempotency	Processing the same chunk twice produces the same result (critical for retries)